



UNIVERSITÀ DI PARMA

Allocazione Dinamica della Memoria e Puntatori

*Memoria est thesaurus omnium rerum et
custos*

De Oratore, Cicerone, I sec a.C.

- Limite array
- Variable Length Array
- I puntatori
- Il tipo void
- Funzioni di gestione memoria
- Errori Comuni

SUMMARY



- Fino ad ora abbiamo usato array con dimensioni costanti
- Ipotesi su quantità dati da gestire
 - Spreco memoria
 - Possibilità di errori → buffer overrun
- Come risolvere?
 - Variable Length Array (VLA)
 - Allocazione dinamica della memoria



- Il C99 ammette di usare una variabile per la dimensione degli array
- Dimensione array decisa in esecuzione
 - Non in codifica!
- Durata sempre automatica
- Non possibile inizializzazione

- Fino al C99 → dimensione array costante
- Per il C99 → dimensione array anche variabile
- Dopo il C99 → dimensione array variabile opzionale
- C++? → ad oggi non previsti nello standard

- Pro
 - Sintassi semplice
- Contro
 - Non portabile
 - C++
 - Non posso dinamicamente cambiare dimensione array
 - Non posso liberare la memoria in uso
 - Dimensioni spesso limitate
 - Durata automatica!

- Il C fornisce primitive di allocazione
 - Sintassi diversa
 - Più flessibili rispetto VLA
- Ma prima dobbiamo introdurre altri elementi
 - Puntatori
 - void
 - size_t

- Principali primitive

```
void *malloc(size_t size);
```

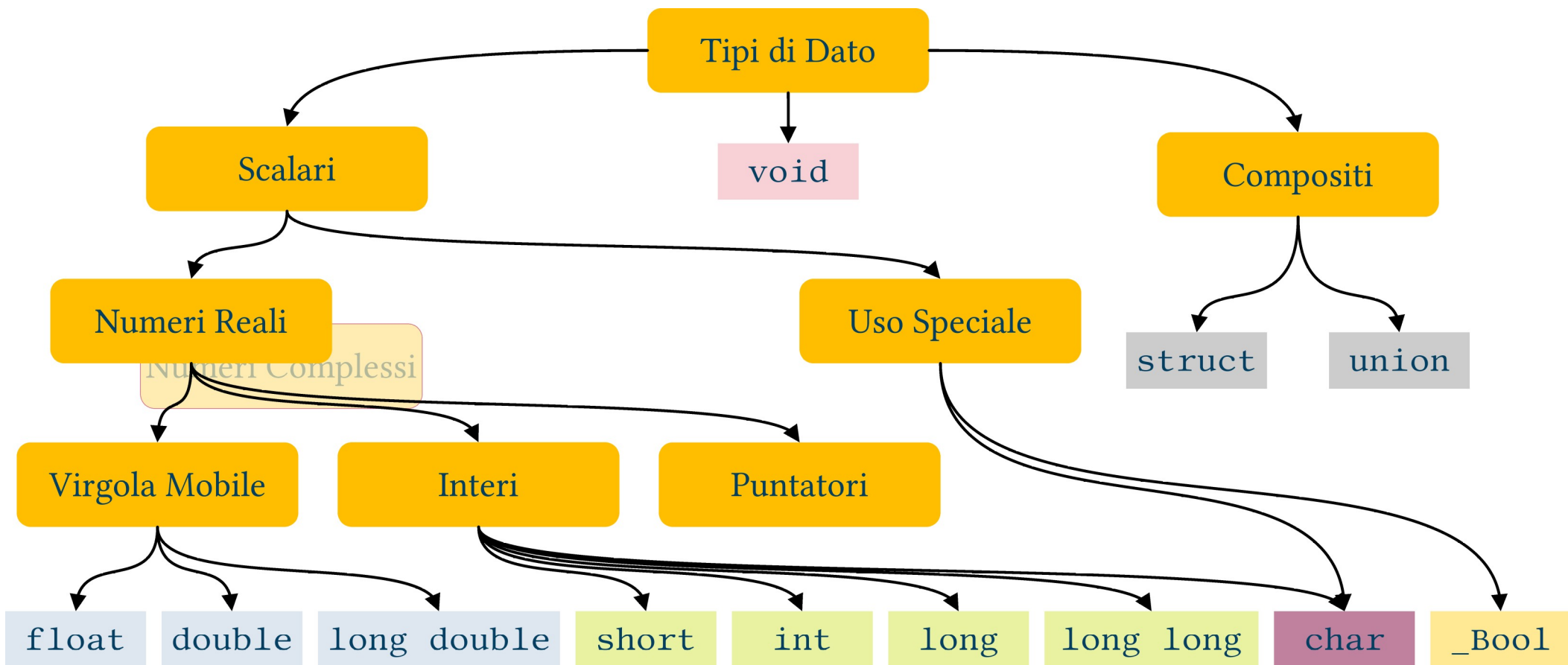
```
void free(void *ptr);
```

```
void *calloc(size_t nmemb, size_t size);
```

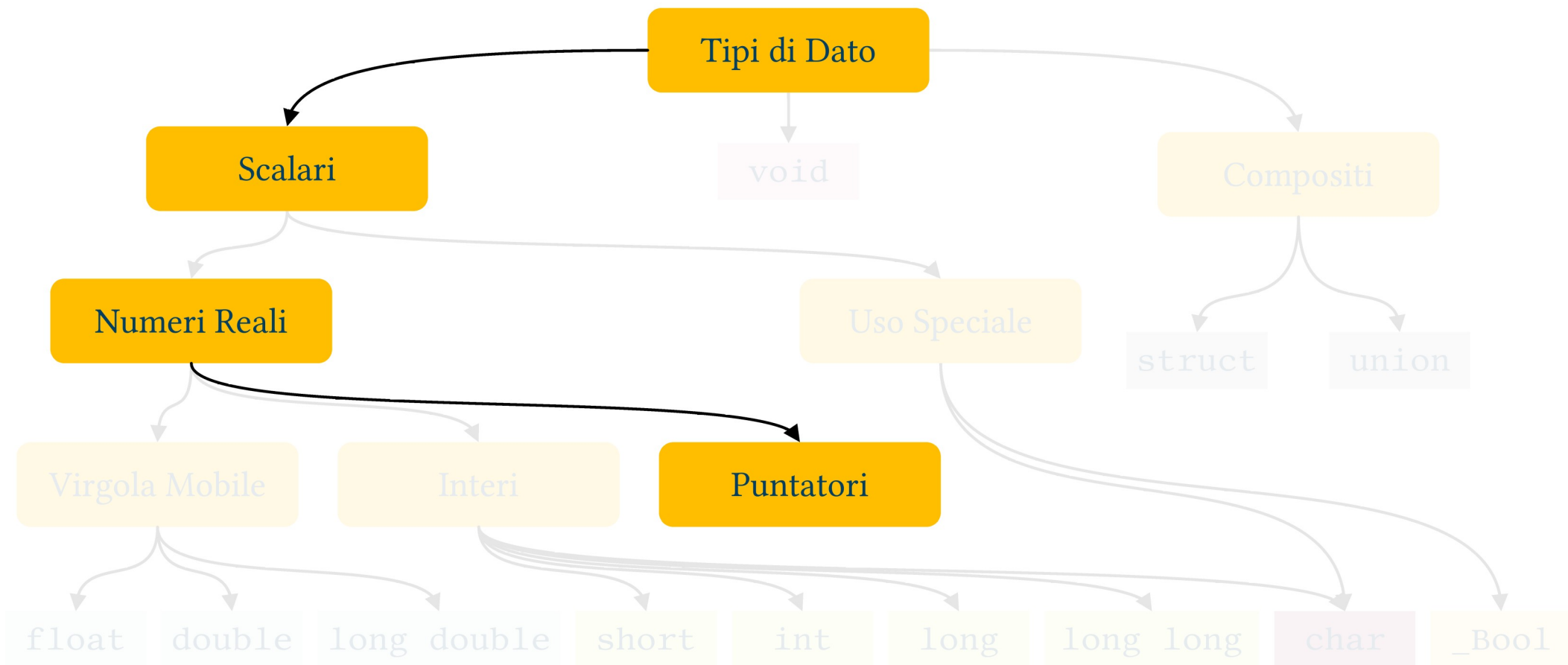
```
void *realloc(void *ptr, size_t size);
```

- Non abbiamo ancora tutti gli elementi per capirle
 - void? *? size_t?

C99 Albero dei Tipi di Dato



C99 Albero dei Tipi di Dato



- Formally, scalar variables
 - More precisely, integer numbers → special meaning
- Two main uses:
 - Dealing with dynamic memory management
 - We are going to discuss this in the following
 - To define functions that can have “side effects”
 - We will be back on this when speaking about functions

- Variabile → spazio in memoria con nome
 - Dimensione
 - Indirizzo
- Esistono casi in cui indirizzo può servire
- Come lo gestisco? → puntatore!
- Un puntatore è un tipo di variabile usato per memorizzare indirizzo di altri dati o elementi del programma

- Come ricavo indirizzo memoria di una variabile?
 - Operatore & per variabili singole
 - Niente per array
 - `printf("L'indirizzo della variabile a e' %p\n", &a);`
 - `printf("L'indirizzo dell'array n e' %p\n", n);`
- Lo posso memorizzare in una variabile puntatore:
 - `int a;       // variabile di tipo int`
 - `int *x;     // puntatore a un dato di tipo int`
 - `x = &a;    // ora x contiene indirizzo di a`

- Per definire un puntatore si usa il carattere ‘*’ prima del nome

```
int *p;    // puntatore a dato di tipo int
```

```
float *x; // puntatore a dato di tipo float
```

```
long a, *b; // solo b è un puntatore!
```

- Dato un puntatore che contiene l'indirizzo di un dato come vi accedo?
- Operatore '*'

```
int a;
```

```
...
```

```
int *x=&a;
```

```
printf("All'indirizzo %p e' memorizzato l'int %d\n", x, *x);
```

Pointers: syntax recap

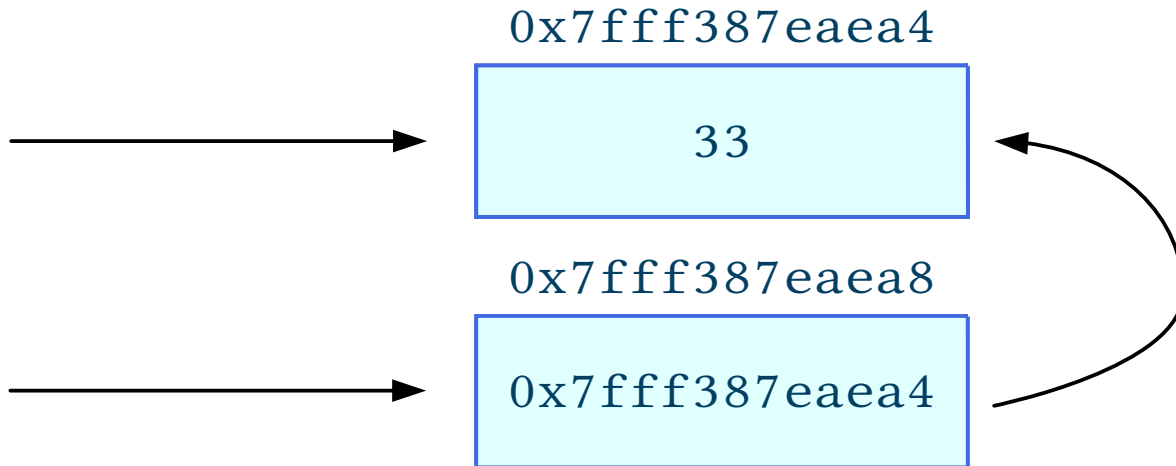
```
int var;
```

```
var = 10;
```

```
int *ptr;
```

```
ptr = &var;
```

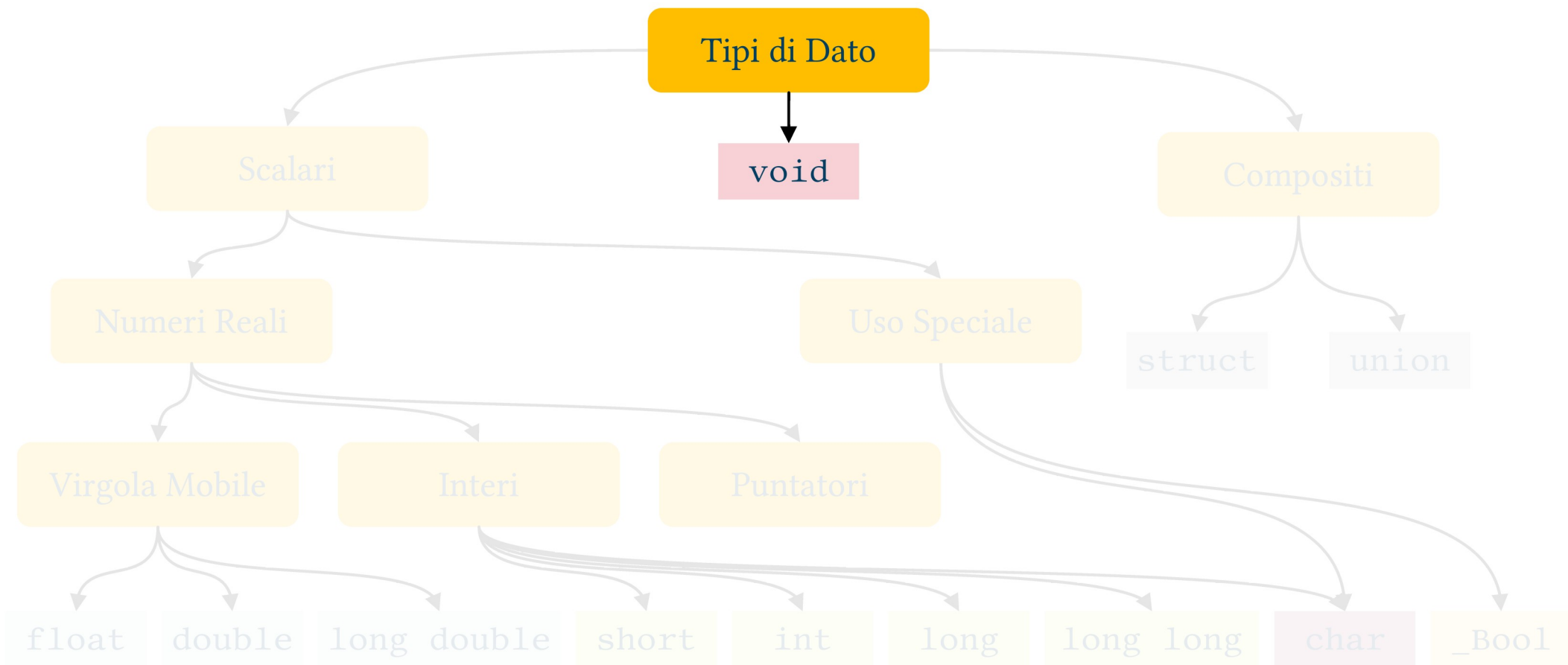
```
*ptr = 33;
```



- Posso ritenere array come puntatori costanti
- Sintassi interscambiabile
 - In particolare []
 - Eccezione sizeof()

- Valore convenzionale usato per indicare indirizzo non valido
- Uso tipico
 - Funzioni che restituiscono indirizzo, se falliscono o non trovano risultato utile restituiscono NULL
 - Come argomento per alcune funzioni

C99 Albero dei Tipi di Dato



- È un *non tipo*
- Non usabile per definizione variabili
 - Funzioni
 - No value
 - No parameters
 - Puntatori
 - No type

- Tecnicamente non è un tipo di dato
- Alias del tipo di dato in grado di contenere la dimensione in byte massima gestibile dal sistema
 - Tipo restituito da sizeof()
- unsigned
- Nei sistemi 64 bit di solito unsigned long long o unsigned long
 - Per stampare %zt (C99, in laboratorio non funziona → %ld)

- Principali primitive

```
void *malloc(size_t size);
```

```
void free(void *ptr);
```

```
void *calloc(size_t nmemb, size_t size);
```

```
void *realloc(void *ptr, size_t size);
```

- Sintassi adesso spiegabile

```
void *malloc(size_t size);
```

- Funzione
 - Prende in ingresso → quantità di memoria
 - Alloca tale quantità di memoria
 - Se possibile
 - Restituisce → indirizzo del buffer allocato
 - NULL se non possibile allocare

- Funzione
 - Prende in ingresso → indirizzo buffer allocato
 - “Disalloca” la memoria
 - Non restituisce niente
- In C non esiste Garbage Collector
 - Se non libero memoria → occupata per sempre!




```
void *calloc(size_t nmemb, size_t size);
```

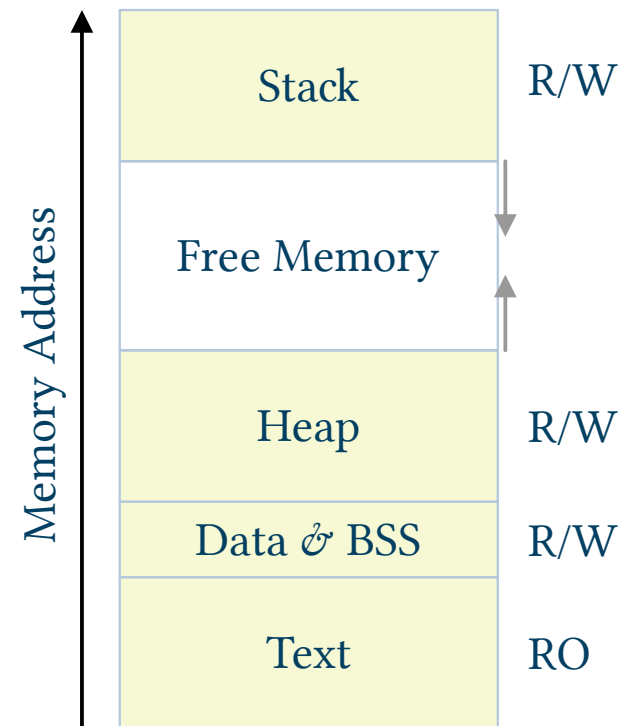
- Funzione
 - Prende in ingresso → numero elementi e loro dimensione
 - Alloca tale “array”
 - Se possibile
 - Lo inizializza a 0
 - Restituisce → indirizzo del buffer allocato
 - NULL se non possibile allocare

```
void *realloc(void *ptr, size_t size);
```

- Funzione

- Prende in ingresso → buffer da ridimensionare, nuova dimensione
- Cambia dimensione buffer
- Ricopia dati
- Restituisce → nuovo indirizzo del buffer allocato
 - NULL se non possibile ridimensionare

- Diverse aree
 - Stack (.stack) → variabili locali, VLA
 - Heap (.heap) → buffer allocati (malloc()...)
 - Code (.text) → codice
 - Data → variabili globali o statiche
 - .data & .bss



- Tipo di dato scalare $\rightarrow$ possibile utilizzo operatori $+$ e $-$
 - Ed equivalenti ($++$, $--$, $+=$ ecc.)
- Risultato dipendente da tipo di dato puntato
- Sommare x non significa incrementare indirizzo di x byte
 - Ma di x “posizioni”

- Se il puntatore punta al tipo di dato `<type>` sommare `x` equivale a sommare `x` volte i byte occupati da `<type>`
- In pratica si usa solo se puntatore contiene indirizzo array
- Di fatto implicita nella sintassi `x[y]`
 - $x[y] \rightarrow *(x+y)$

- Come per array facile commettere errori
 - Dimensioni buffer
 - Durata variabili
 - Perdita informazioni su aree allocate



- Come per gli array rischio di andare oltre i limiti di quanto allocato
- Inoltre → puntatore non inizializzato o NULL



- malloc()/calloc() restituiscono indirizzo di quanto allocato
 - Serve per usare area memoria
- E se perdo tale indirizzo?
 - Memoria diventa inutilizzabile
 - Ma comunque “in uso”
- Cause
 - Errori Assegnazione
 - Durate Automatiche



- È il duale del precedente
- Non perdo l'informazione di dove si trova il buffer ma...
 - Il buffer non è più valido
- Cause
 - Durata automatica
 - Errori uso free()



- Also pointer variables feature a memory address
- Can we store that address?
- Yes → pointers to pointers
- You need to use a double `**` in definitions
- i.e. `int **p2p; //p2p is a double pointer`
- Also pointer to pointer variables feature a memory address...
 - You can use a lot of `*`...



- Do you remember the main() function?
 - `int main(int argv, char **argc);`
- Why we need them?
 - When dealing with functions we want side effects, or
 - In order to dynamically allocate/manage multidimensional arrays
 - i.e. like in the case of the main() function

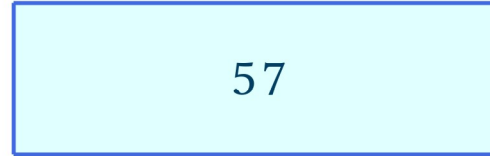


Pointers to Pointers: syntax

```
int var;
```



0x7fff387eaea4



```
var = 10;
```

0x7fff387eaea8

```
int *ptr;
```



0x7fff387eaea4



```
ptr = &var;
```

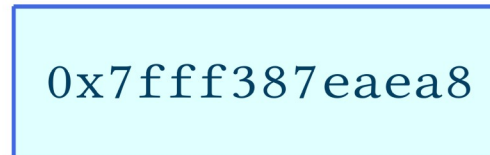
```
*ptr = 33;
```

0x7fff387eaeb0

```
int **dptr;
```



0x7fff387eaea8



```
dptr = &ptr;
```

```
**dptr = 57; // note the double **
```



Pointers to Pointers: reason why

- Double pointers, triple pointers etc.
- Again, two main uses:
- Dealing with dynamic memory management
 - To dynamically manage multiple dimensions arrays
- To define functions that can have “side effects”



Pointers to Pointers: multidim arrays recap

- Just consider a bi-dimensional array
 - For sake of simplicity

- We can define it as

```
float b[33][55];
```

- To access a specific element, we can use a syntax like:

```
b[17][34] = ... // we MUST use [][] and indexes
```

- To access array address

```
printf("the address is %p\n", b) // we have to omit []
```

Pointers to Pointers: multidim arrays recap

```
float b[33][55];
```

- Can we use a single index with a bi-dimensional array?
- i.e. it is correct to use and what is the meaning of

```
b[17] ??
```

- When we omit some dimension, this always represents an address. Which one?
- We can see b as an array of arrays, namely that is the address of the 18th row

```
printf("the address of 18th row is %p\n", b[17])
```

- In order to dynamically manage a multi-dim array we have to use pointers to pointers → same “dimensionality”
- Lets’ consider bi-dimensional array → double pointer
- Using a double pointer we initially allocate an array of pointers to deal with “rows”
- Then we allocate each pointer element to deal with “columns”

Dynamically manage multi-dim arrays: example

```
// how to obtain the same as int a[55][77]?  
  
// define double pointer to deal with a bi-dim array  
  
int **matrix  
  
  
// initially allocate 1 pointer for each row  
matrix = malloc(55 * sizeof(int *)); // note the "int *"  
  
  
// then allocate each row  
for(int i = 0; i < 77; ++i)  
    matrix[i] = malloc(77 * sizeof(int));  
  
  
// now we can use matrix using the, easier, array syntax  
matrix[17][34] = 99;
```



UNIVERSITÀ DI PARMA

Allocazione Dinamica della Memoria e Puntatori



KEEP
CALM
IT'S
QUESTION
TIME

Ma noi paghiamo i byte? No, e allora...

Z, VisLab srl, 2014